Python | Machine Learning | AI 🚀
@pycode.hubb

PYTHON FOR BEGINNERS: EASY AND DETAILED GUIDE

by Pycode.Hubb

python™

Disclaimer:

The content provided in these notes is intended for educational purposes only. While every effort has been made to ensure the accuracy and reliability of the information presented, the author and publisher assume no responsibility for errors, omissions, or any outcomes related to the use of this material. The examples and explanations are simplified to aid understanding and may not cover all possible scenarios or edge cases.

These notes are not a substitute for professional advice or reference materials, and users are encouraged to consult official documentation and other reputable sources for comprehensive and up-to-date information. The author and publisher disclaim any liability for any damages or losses incurred by the use or misuse of the content provided.

By using these notes, you agree to indemnify and hold harmless the author and publisher from any and all claims, liabilities, or expenses arising from their use.

Copyright Notice:

TABLE OF CONTENTS

python™

TABLE OF CONTENTS

python™

TABLE OF CONTENTS

python™

Chapter 1

INTRODUCTION TO PYTHON

python™

1.1 What is Python?

Python is a **high-level programming language** known for its readability and simplicity in implementation. Being open-source, Python is **freely available** for both personal and commercial use.

Its built-in data structures, dynamic typing, and dynamic binding facilitate **rapid application development and scripting**. Python's **user-friendly syntax** prioritizes readability, which lowers maintenance costs.

Additionally, Python supports modules and packages to promote code modularity and reuse, enhancing efficiency in software development.

1.2 Features of Python

1. Easy to Learn: Python has a simple syntax and structure, making it beginner-friendly and quick to start coding.

2. Easy to Code: It minimizes coding efforts with a rich library ecosystem, enabling fast development of complex applications.

3. Interpreted Language: Executes code line by line, aiding real-time debugging and error fixing.

4. Dynamic Typed: Adapts flexibly to variable data types at runtime, supporting agile and adaptable coding practices.

5. Free and Open Source: Anyone can use, modify, and distribute Python without cost, ideal for cost-effective software development.

6**. Object-Oriented:** Supports principles like encapsulation and inheritance for reusable and maintainable code.

7**. Cross-Platform:** Runs on Windows, Linux, and macOS, facilitating development across different operating systems.

8**. Extensive Features:** Supports multi-threading, networking, and database connectivity for building enterprise-level applications.

9**. High-Level Language:** Closer to human language, enhancing code readability and ease of maintenance.

10**. Database Support:** Built-in support for databases like MySQL, PostgreSQL, and SQLite simplifies data storage and retrieval.

11. GUI Programming: Offers GUI libraries like Tkinter, PyQt, and wxPython for creating graphical interfaces.

12. Large Standard Library: Includes modules for web development, networking, and data processing, reducing the need for custom code.

1.3 Applications of Python

Python's **versatility and user-friendly design** make it indispensable across various domains, offering numerous practical applications in the real world. Here are some of them:

1. Data Analysis and Visualization

Python is a powerful tool for analyzing and visualizing data, which is essential for modern businesses.

It offers a vast standard library and specialized modules tailored for data analysis. It includes popular libraries like Pandas and NumPy for tasks such as cleaning data, exploring statistics, and uncovering trends.

Python also supports numerous other libraries for tasks like data visualization (Matplotlib, Seaborn, Plotly, Bokeh), web scraping, and hypothesis testing, making it versatile for a wide range of data-related jobs.

2. Web Development

Python is widely used in web development, particularly for building the backend of websites. While HTML, CSS, and JavaScript handle the front end (visible to users), Python excels in the back end (invisible part).

Python's strength lies in frameworks like Django and Flask, which offer specialized modules for tasks such as data sharing, server-side processing, database management, URL routing, content management, and security.

Noteworthy websites and applications built with Python include **Google, Facebook, Instagram, YouTube, Dropbox, and Reddit**, highlighting its robustness and scalability in handling large-scale web applications.

3. Machine Learning and Artificial Intelligence

Python is widely favored for machine learning and artificial intelligence due to its user-friendly syntax and powerful libraries. It offers essential tools like NumPy, Pandas, and Scikit-Learn for data manipulation and modeling.

Frameworks like TensorFlow and PyTorch are used extensively for building neural networks and deep learning models. Python's versatility, community support, and integration capabilities make it ideal for developing and deploying AI applications across different platforms.

4. Task Automation/Scripting

Python is ideal for automating repetitive tasks through scripting. This involves writing programs to automate actions such as file and folder manipulation, creation, renaming, conversion, and more complex operations like

error checking and text pattern recognition. Python automation extends to tasks like internet data retrieval, form completion and submission, and regular notifications or email sending.

1.4 Installation

Now, let's learn how to install the latest versions of Python and VS Code on your Windows system. Since I have a Windows system, I'll guide you through the installation process for Windows only. If you're using macOS or Linux, please refer to YouTube tutorials or blog posts for the appropriate steps.

FOR WINDOWS:

1. PYTHON INSTALLATION

To download and install Python, follow these steps:

1. Visit python.org, the official Python website.

2. Navigate to *python.org/downloads/windows/*.

3. Choose the appropriate installer for your system: 32-bit or 64-bit.

4. After downloading, locate the installer file and run it.

5. Follow the installation wizard's instructions. Ensure to check "***Add Python X.X to PATH***" for easy command-line access.

6. Once installation is complete, open your command prompt or terminal and type `***python --version***` to verify the installed Python version.

FOR WINDOWS:

2. VS CODE INSTALLATION

To download and install VS Code, follow these steps:

1. Go to the https://code.visualstudio.com/ and click "**Download for Windows.**"

2. Run the downloaded ***VSCodeSetup.exe***.

3. Accept the license agreement and click "**Next**."

4. Choose or confirm the installation location and click "**Next**."

5. In the additional tasks section, ensure to **check all the boxes** as they are essential.

6. Click "**install**".

7. Check "**Launch Visual Studio Code**" and click "Finish".

Now install the necessary extensions in VS Code: '**Code Runner**' by Jun Han and '**Python' by Microsoft**.

1.5 First Python program

Your first Python program will be a simple one to get you started. Here's how to do it:

1. Navigate to the folder where you want to save your program.

2. Right-click in the folder, select **"Open with Code"** from the context menu.

3. Visual Studio Code (VS Code) will open with the selected folder.

4. Inside VS Code, create a new file named `***first_program.py***`.

5. Type the following code into the file:

```
print("Hello, world!")
```

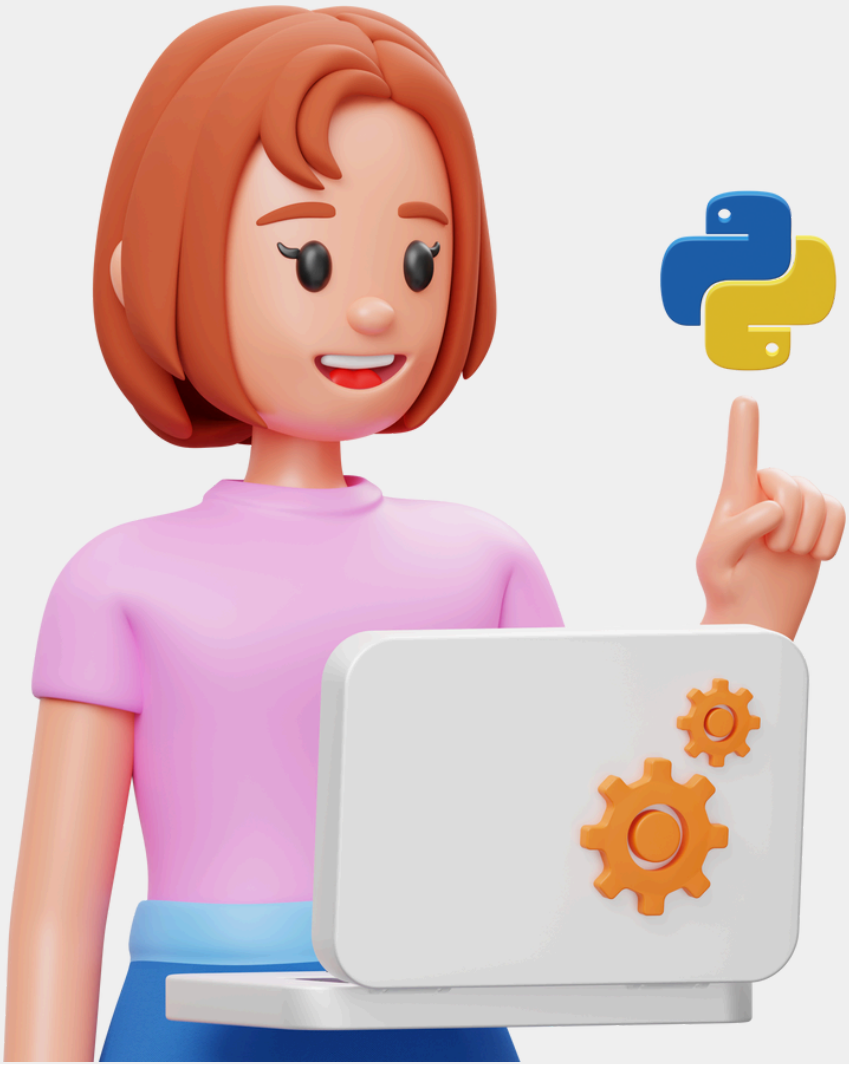
6. Save the file (Ctrl + S) and run it by clicking on the "Run" button in VS Code, or use the shortcut Ctrl + Alt + N.

7. Check the terminal output in VS Code; you should see **"Hello, World"** printed.

Let’s break down the code:

- **print:** This is a built-in Python function that outputs data to the screen. In this case, it's used to display text.
- **"Hello, world!":** This is a string, a type of data in Python that represents text. The text inside the quotes is what will be displayed when the program runs.

Putting it all together, **print("Hello, world!")** tells Python to display the text "Hello, world!" on the screen.



Chapter 2

MODULES, COMMENTS, & PIP

python™

2.1 Module

Modules are like **toolboxes in Python**. They are files that contain extra tools **(functions, classes, and variables)** that you can use in your programs. Think of them as pre-made sets of instructions that can help you do specific tasks without writing all the code yourself.

For example, if you need to work with dates and times, you can **import** the **datetime module**. This module contains tools (like functions and classes) that make it easier to work with dates and times in your Python programs.

By importing datetime, you gain access to all the tools (like **datetime.datetime.now()** to **get the *current date and time***).

Why Use Modules?

- **To reuse code:** Instead of writing the same code again and again, you can write it once in a module and use it whenever you need it.
- **To organize your code:** Keep related code together in one file for better organization.
- **To use other people's code:** Python has many built-in modules and libraries created by other people that you can use in your programs.

Types of Modules

In Python, modules can be broadly categorized into three types based on where they come from and how they are used:

1. Built-in Modules: These are modules that are part of the Python Standard Library. They come pre-installed with Python, so you can use them without needing to install anything extra. For example ***math, datetime, and random*** modules.

2. Third-Party Modules: These modules are created by developers outside of the Python Standard Library. They extend Python's functionality beyond what's available in the standard modules.

You need to install third-party modules using tools like **Pip** before you can use them in your code. Examples include ***requests*** for making HTTP requests and ***NumPy*** for numerical computations.

3. Custom Modules: These are modules that you create yourself. They allow you to organize your code into reusable components. You can define functions, classes, and variables in these modules and then import and use them in your Python programs.

2.2 Comments

Comments are notes you add within your code **to explain what the code does**. They start with the `#` symbol and are ignored by the Python interpreter when your code runs.

Comments help you and others understand the purpose of different parts of your code without affecting how the program works.

They're like little reminders or explanations you leave for yourself or anyone reading your code.

Types of Comments

There are two types of comments in Python, which are explained below:

1. Single-line comments: These comments start with # and extend to the end of the line. They are used for short explanations or notes on a single line of code.

For Example:

```
# This is a single-line comment
x = 5  # This is another single-line comment
```

2. Multi-line comments (docstrings): While Python doesn't have a specific syntax for multi-line comments, docstrings serve a similar purpose.

Docstrings are enclosed in triple quotes (''' ''') and are typically used to describe functions, modules, or classes.

They can span multiple lines and are used to generate documentation automatically.

For Example:

```
'''
This function adds two numbers together.
Parameters:
a (int): The first number.
b (int): The second number.

Returns:
int: The sum of a and b.
'''
def add_numbers(a, b):
    return a + b
```

These two types of comments help you document your code effectively, making it easier to understand and maintain.

2.3 Pip

In Python, pip is a tool that **helps you install and manage additional libraries and packages** that extend the functionality of Python.

It stands for **"Python Package Index."** with Pip, you can easily download and install libraries from the Python Package Index **(PyPI)** or other sources.

It simplifies the process of adding new features or tools to your Python environment, making it a valuable tool for developers.

Example of how you can use pip in Python:

1. Installing a Package:

Suppose you want to install the requests library, which is commonly used for making HTTP requests in Python.

In your terminal type:

```
pip install requests
```

This command tells pip to **download and install** the **requests library** from PyPI (Python Package Index).

2. Listing Installed Packages:

You can also use pip to list all installed packages in your Python environment. Type:

```
pip list
```

This command will display a list of all packages along with their currently installed versions.

3. Upgrading a Package:

If you want to upgrade a package to its latest version, you can use pip as well. For example, to upgrade the requests library, you would type:

```
pip install --upgrade requests
```

This command will upgrade the requests library to the latest available version.

Chapter 3

VARIABLES, DATA TYPES, KEYWORDS & OPERATORS

python™

3.1 Variables in Python

Variables are like containers that hold data. They have names that you choose, and you can put different types of information in them, like **numbers, text, or lists**.

For example:

```
age = 45
name = "Rahul"

print(age) # Output will be 45
print(name) # Output will be Rahul
```

3.2 Data Types

In Python, data types define the type of data a variable can hold. Here are the main built-in data types:

1. Integer (int): Represents whole numbers without any decimal point. Example: 10, -5, 1000.

2. Float (float): Represents real numbers with a decimal point. Example: 3.14, 2.5, -0.1.

3. String (str): Represents text and is enclosed in single (') or double (") quotes. Example: 'hello' & "Python is awesome".

4. Boolean (bool): Represents truth values, either True or False. Used for logical operations and comparisons.

5. List: Represents an ordered collection of items. Lists are mutable (can be changed after creation). Example: [1, 2, 3, 4], ['apple', 'banana', 'cherry'].

6. Tuple: Similar to lists but immutable (cannot be changed after creation). Example: (1, 2, 3), ('a', 'b', 'c').

7. Dictionary (dict): Represents a collection of key-value pairs. Keys are unique within a dictionary and are used to access values. Example: {'name': 'John', 'age': 30, 'city': 'New York'}.

8. Set: Represents an unordered collection of unique items. Example: {1, 2, 3, 4}, {'apple', 'banana', 'cherry'}.

9. NoneType (None): Represents the absence of a value or a null value.

Python is dynamically typed, meaning you don't need to explicitly declare the data type of a variable when you assign a value to it.

The interpreter determines the type based on the value assigned.

Getting the Data Type

You can determine the data type of any object in Python using the **type()** function.

For example:

- Let's print the **data type of variable a**:

```
a = 45
print(type(a)) # Output will be <class 'int'>
```

Casting

In Python, casting refers to the ***process of converting a variable from one data type to another***. This is useful when you need to change how a value is interpreted or used in your program.

Here's a simple example:

```
# Convert a string to an integer
num_str = "10"
num_int = int(num_str)

# Convert an int to str
num_int = 20
num_str = str(num_int)
```

3.3 Keywords

Keywords in Python are **reserved words** that have **special meanings and purposes**. They are used to define the syntax and structure of Python programming language. Keywords **cannot be used as identifiers** (names for variables, functions, etc.) in your program.

Here is a list of reserved keywords in Python:

Flase	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

3.4 Identifiers in Python

In Python, An identifier is a **user-defined name** for a variable, function, class, module, or other objects in Python.

Identifiers can include letters, digits, and underscores, but they **must start with a letter or an underscore**.

They are **case-sensitive**, meaning num, Num, and NUM are considered different identifiers.

Using meaningful names for identifiers is good programming practice, as it makes the code easier to understand.

Rules for Naming Python Identifiers

- It cannot be a reserved python keyword.
- It should not contain white space.
- It can be a combination of A-Z, a-z, 0-9, or underscore.
- It should start with an alphabet character or an underscore (_).
- It should not contain any special character other than an underscore (_).

3.5 Operators

In Python, **operators are symbols** that enable you to **perform operations on variables and values**. There are multiple types of Python operators, which we will explore one by one.

Python Arithmetic Operators

Arithmetic operators are **used with numeric values** to perform common mathematical operations.

Operator	Name	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modules	a % b
**	Exponentiation	a ** b
//	Floor division	a // b

Python Assignment Operators

Assignment operators are used to assign values to variables.

Operators	Example	Same as
=	a = 5	a = 5
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 8
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5
//=	a //= 5	a = a // 5
**=	a **= 5	a = a ** 5
&=	a &= 5	a = a & 5
|=	a |= 5	a = a | 5
^=	a ^= 5	a = a ^ 5
>>=	a >>= 5	a = a >> 5
<<=	a <<= 5	a = a << 5

Python Comparison Operators

Comparison Operators in Python are used to compare two values.

Operator	Name	Example
==	Equal	a == b
!=	Not Equal	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or Equal to	a >= b
<=	Less than or equal to	a <= b

Python Logical Operator

used to combine conditional statements.

Operator	Description	Example
and	Returns True if both statements are true	a < 4 and a < 4
or	Returns True if one of the statements is true	a < 4 or a < 6
not	Reserve the result, return false if the result is true	not(a < 4 and a < 20)

Python Membership Operators

Membership operators are used to test if a sequence is presented in an object.

Operators	Description	Example
in	Returns True if a sequence with the specified value is present in the project	a in b
not in	Returns True if a sequence with the specified value is not present in the object	a not in b

Python Identity Operators

Identify Operators are used to compare the objects not if they are equal, but if they are the same object, with the same memory location.

Operator	Description	Example
is	Return True if both variables are the same	a is b
is not	Return True if both variables are not the same objects	a is not b

Python Bitwise Operators

BitWise operators are used to compare (binary) numbers.

Operators	Name	Description
&	And	Sets each bit to 1 if both bits are 1
|	Or	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	Not	Inverts all the bits
<<	Zero file left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit

Chapter 4

DETAILED OVERVIEW OF PYTHON DATA TYPES

python™

4.1 Python Numbers

There are three numeric types in Python:

- **int:** Integer numbers without decimals.
- **float:** Real numbers with decimals.
- **complex:** Numbers with a real and imaginary part.

Examples:

```
a = 1 # int
b = 4.9 # float
c = 3 + 2j # complex
```

Integers (int):

Represents integer **numbers without decimals.**

For example, a = 10

In this example, **a** is assigned the **integer value 10.**

Float

Represents real numbers with decimals.

For example, *b = 3.14*

In this example, **b** is assigned the float value **3.14**.

Complex

Represents numbers with a real and imaginary part.

For example, **c = 2 + 3j**

In this example, **c** is assigned the complex number **2 + 3j**.

Type Conversion

In Python, you can convert between different data types using the **int(), float(),** and **complex()** functions.

Examples:

1. Converting to int:

```
num_float = 3.14
num_int = int(num_float) # Float to int
print(num_int) # Output 3
```

2. Converting to float:

```
num_int = 5
num_float = float(num_int) # int to float
print(num_float) # Output: 5.0
```

3. Converting to complex:

```
num = 2
num_complex = complex(num, 3) # int to complex
print(num_complex) # Output: (2+3j)
```

4.2 Python Strings

A string in Python is a sequence of characters. Which can be letters, numbers, symbols, or spaces.

You can create a string by enclosing characters in single quotes '', double quotes " ", or triple quotes ''' '''.

For Example:

```
single_quote_string = 'Hello, World!'
double_quote_string = "Python is fun!"
triple_quote_string = '''This is a multiline
string example.'''
```

String Methods

Python has a set of built-in methods that you can use on strings.

- **capitalize():** Converts the first character to upper case.
- **casefold():** Converts string into lower case.
- **center():** Returns a centered string.
- **count():** Returns the number of times a specified value.
- **endswith():** Returns true if the string ends with the specified value.
- **find():** Searches the string value and returns the position of where it was found.
- **format():** Formats specified values in a string.
- **index():** Searches the string for a specified value and returns the position of where it was found.
- **isalnum():** Returns True if all characters in the string are alphanumeric.
- **isascii():** Returns True if all characters in the string are ascii characters.
- **join():** Joins the elements of an iterable to the end of the string
- **ljust():** Returns a left justified version of the string.

- **isdecimal():** Returns True if all characters in the string are decimals.
- **isdigit():** Returns True if all characters in the string are digits.
- **isidentifier():** Returns True if the string is an identifier.
- **islower():** Returns True if all characters in the string are lower case.
- **isprintable():** Returns True if all characters in the string are printable.
- **lower():** Converts a string into lower case.
- **partition():** Returns a tuple where the string is parted into three parts.
- **replace():** Returns a string where a specified value is replaced with a specified value.
- **rfind():** Searches the string for a specified value and returns the last position of where it was found.
- **rpartition():** Returns a tuple where the string is parted into three parts.
- **startswith():** Returns true if the string starts with the specified value.

4.3 Python Booleans

A Boolean is a data type that can only have one of two values: **True** or **False**. Booleans are often used in conditional statements and comparisons. For examples:

1. Comparisons:

```
print(5 > 3) # Output: True
print(5 == 3) # Output: False
print(5 < 3) # Output: False
```

2. Boolean Operations:

- **and:** Returns True if both statements are true.
- **or:** Returns True if at least one statement is true.
- **not:** Reverses the boolean value.

```
print(True and False) # Output: False
print(True or False) # Output: True
print(not True) # Output: False
```

3. In Conditional Statements

```
is_sunny = True
if is_sunny:
    print("Let's go to the park!")

# This will print because is_sunny is True.
```

4. Boolean Functions:

Many functions return a Boolean value:

```
print(bool(1)) # Output: True
print(bool(0)) # Output: False
print(bool("")) # Output: False
print(bool("Hello")) # Output: True
```

Booleans are essential for making decisions in your code, helping control the flow based on conditions.

4.4 Python List

A list is a collection of items in a particular order. They can contain items of different data types: numbers, strings, other, etc. Lists are defined using **square brackets [ ]**.

Creating a List

```
fruits = ['apple', 'banana', 'cherry']
numbers = [1, 2, 3, 4, 5]
mixed = [1, 'apple', 3.5, True]
```

Accessing List Items

You can access items in a list by their index, starting from 0.

```
print(fruits[0]) # Output: 'apple'
print(fruits[1]) # Output: 'banana'
```

Modifying List Items

You can change the value of a list item by accessing its index.

```
fruits = ['apple', 'banana', 'cherry']
fruits[1] = 'blueberry'
print(fruits)
# Output: ['apple', 'blueberry', 'cherry']
```

Removing Items from a List

- **remove():** Removes the first occurrence of a specified item.

```
fruits = ['apple', 'banana', 'blueberry',
'cherry', 'date']
fruits.remove('banana')
print(fruits) # Output: ['apple', 'blueberry',
'cherry', 'date']
```

- **pop():** Removes an item at a specified index (or the last item if index is not specified).

```
fruits = ['apple', 'blueberry', 'cherry',
'date']
fruits.pop(2)
print(fruits)
# Output: ['apple', 'blueberry', 'date']
```

- **del():** Deletes an item at a specified index.

```
fruits = ['apple', 'blueberry', 'date']
del fruits[1]
print(fruits) # Output: ['apple', 'date']
```

List Operations

- **len():** Returns the number of items in a list.

```
fruits = ['apple', 'date']
print(len(fruits)) # Output: 2
```

- **sort():** Sorts the list in ascending order.

```
fruits = ['date', 'apple']
fruits.sort()
print(fruits) # Output: ['apple', 'date']
```

- **reverse():** Reverses the order of the list.

```
fruits = ['apple', 'date']
fruits.reverse()
print(fruits) # Output: ['date', 'apple']
```

Looping Through a List

You can loop through the items in a list using a for loop.

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# cherry
```

Lists are versatile and useful for storing and manipulating collections of items in Python.

4.5 Tuple in Python

A tuple is a collection of items, similar to a list. They are ordered, and their elements are indexed starting from 0. Tuples are defined using **parentheses ()**.

Creating a Tuple

```
fruits = ('apple', 'banana', 'cherry')
numbers = (1, 2, 3, 4, 5)
mixed = (1, 'apple', 3.5, True)
```

Accessing Tuple Items

You can access items in a tuple by their index, just like in a list.

```
fruits = ('apple', 'banana', 'cherry')
print(fruits[0]) # Output: 'apple'
print(fruits[1]) # Output: 'banana'
```

Immutable Nature

- Tuples are immutable, meaning their elements cannot be changed or modified after the tuple is created.
- However, you can create a new tuple with updated elements.

```
# Attempting to modify a tuple will result in an error
fruits[1] = 'blueberry'

# This will raise an error: TypeError: 'tuple' object does not support item assignment

# You can create a new tuple with the updated value
new_fruits = fruits + ('blueberry',)
print(new_fruits)

# Output: ('apple', 'banana', 'cherry', 'blueberry')
```

Advantages of Tuples

- **Faster:** Tuples are generally faster than lists for accessing data.
- **Safe:** Since tuples are immutable, their contents cannot be accidentally changed.

When to Use Tuples

Use tuples when you have a collection of items that should not change, such as coordinates, configuration settings, or constant values.

Tuple Operations

- **len():** Returns the number of items in a tuple.

```
fruits = ('apple', 'banana', 'cherry')
print(len(fruits)) # Output: 3
```

- **index():** Returns the index of a specified item in the tuple.

```
fruits = ('apple', 'banana', 'cherry')
print(fruits.index('cherry')) #Output: 2
```

- **count():** Returns the number of times a specified value occurs in the tuple.

```
fruits = ('apple', 'banana', 'cherry')
print(fruits.count('apple'))

# Output: 1
```

Looping Through a Tuple

You can loop through the items in a tuple using a for loop.

```
fruits = ('apple', 'banana', 'cherry')

for fruit in fruits:
    print(fruit)
# Output:
# apple
# banana
# cherry
```

Tuples are useful when you want to store a collection of items that should not change throughout the program execution.

4.6 Dictionary in Python

A dictionary is a **collection of key-value pairs**. Each key is unique within a dictionary, and it maps to a corresponding value. They are defined using **curly braces { }**.

Creating a Dictionary

```
# Creating a dict of person's details

person = {
    'name': 'John',
    'age': 30,
    'city': 'New York'
}

# Creating an empty dictionary
empty_dict = {}
```

Accessing Dictionary Items

You can access the value associated with a key in a dictionary using **square brackets [ ]**.

```
print(person['name']) # Output: 'John'
print(person['age']) # Output: 30
```

Modifying Dictionary Items

- You can change the value of a specific key.

```
person = {
    'name': 'John',
    'age': 30,
    'city': 'New York'
}

person['age'] = 32
print(person['age']) # Output: 32
```

Adding Items to a Dictionary

- You can add new key-value pairs to a dictionary.

```
person['job'] = 'Developer'
print(person)

# Output: {'name': 'John', 'age': 32,
'city': 'New York', 'job': 'Developer'}
```

Removing Items from a Dictionary

- You can remove a key-value pair using the del keyword or the pop() method.

```
del person['city']
print(person) # Output: {'name': 'John',
'age': 32, 'job': 'Developer'}

job = person.pop('job')
print(job)  # Output: 'Developer'
print(person) # Output: {'name': 'John',
'age': 32}
```

Dictionary Operations

- **len():** Returns the number of key-value pairs in the dictionary.

```
print(len(person)) # Output: 2
```

- **keys():** Returns a list of all the keys in the dictionary.

```
print(person.keys())
# Output: dict_keys(['name', 'age'])
```

- **values():** Returns a list of all the values in the dictionary.

```
print(person.values())
# Output: dict_values(['John', 32])
```

- **items():** Returns a list of key-value pairs (tuples) in the dictionary.

```
print(person.items())
# Output: dict_items([('name', 'John'),
('age', 32)])
```

Dictionaries are versatile and efficient for storing and retrieving data when you want to associate values with keys.

4.7 Sets in python

A set is an **unordered collection of unique items**. They are defined using **curly braces { }**. Sets do not allow duplicate elements.

Creating a Set

```
fruits = {'apple', 'banana', 'cherry'}
numbers = {1, 2, 3, 4, 5}
mixed = {1, 'apple', 3.5, True}
```

Accessing Set Items

- Since sets are unordered, you cannot access items by index like in lists or tuples.
- You can check membership using in keyword.

```
print('banana' in fruits) # Output: True
```

Adding Items to a Set

- Use the **add()** method to add a single item to a set.

```
fruits.add('orange')
print(fruits)
# Output: {'apple', 'banana', 'cherry',
'orange'}
```

- Use the **update()** method to add multiple items (from another set, list, tuple, etc.) to a set.

```
fruits.update(['pineapple', 'grape'])
print(fruits)

# Output: {'apple', 'banana', 'cherry',
'orange', 'pineapple', 'grape'}
```

Removing Items from a Set

- Use the **remove()** method to remove a specific item from the set. Raises a KeyError if the item is not found.

```
fruits.remove('banana')
print(fruits)
# Output: {'apple', 'cherry', 'orange',
'grape', 'pineapple'}
```

- Use the **clear()** method to remove all items from the set.

```
fruits.clear()
print(fruits) # Output: set()
```

- Use the **pop()** method to remove and return an arbitrary item from the set.
- Use the **discard()** method to remove a specific item from the set without raising an error if the item is not found.

```
fruits.discard('melon')
print(fruits)
# Output: {'apple', 'cherry', 'orange',
'grape', 'pineapple'}
```

Set Operations

- **Union (|):** Returns a set containing all unique items from both sets.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 | set2) # Output: {1, 2, 3, 4, 5}
```

- **Intersection (&):** Returns a set containing common items between both sets.

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
print(set1 & set2) # Output: {2, 3}
```

- **Difference (-):** Returns a set containing items that are only in the first set and not in the second set.
- **Symmetric Difference (^):** Returns a set containing items that are in either set, but not both.

Chapter 5

FLOW CONTROL IN PYTHON

python™

5.1 If Statement

In Python, an **if statement** allows you to execute a code block only if a specified condition is **true**. This feature helps you **control the flow of your program** based on whether conditions are met or not.

We have already seen in previous chapters that, Python supports the usual **logical conditions** from mathematics.

- Equals : **a == b**
- Not Equals : **a != b**
- Less than : **a < b**
- Less than or equal to : **a <= b**
- Greater than : **a > b**
- Greater than or equal to : **a >= b**

These conditions are **used extensively in** if statements and loops to make decisions based on data.

An if statement is **constructed** using the **if** keyword.

It allows you to execute a code block only when a specified condition evaluates to true. Here's a basic example of its usage:

```
if condition:
 # Code to execute if the condition is true
 statement1
 statement2
 # ...
```

In Python:

- Conditions are typically expressed using comparison operators **(==, !=, <, <=, >, >=)** to compare values.
- The if statement checks if a condition evaluates to **True**. If it does, the indented code block following the if statement is executed
- **Indentation** (typically four spaces) is crucial in Python to denote the code block that should be executed if the condition is true.

For example, let's use a condition to check if a number is greater than 10:

```
num = 15

if num > 10:
    print("Number is greater than 10")
```

In this example:

- **num > 10** is the condition.
- Since **num is 15**, which satisfies the condition (***num > 10 is True***), the message ***"Number is greater than 10"*** will be printed.

The if statement is fundamental for making decisions in Python programs, allowing them to respond dynamically based on different conditions.

5.2 If-else statement

In addition to the if statement, Python also provides an **if-else statement**, which allows you to execute one code block if a condition is true and another code block if the condition is false.

This is useful when you want your program to take alternative actions based on whether a condition is met or not.

Here's how the if-else statement works:

```
if condition:
  # Code to execute if the condition is true
  statement
else:
  # Code to execute if the condition is false
  statement
```

- **condition:** This is an expression that evaluates to either ***True or False***. If the condition is True, the indented code block under if is executed. If False, the block under else is executed.

For example, let's modify the previous example to include an else statement to handle the case when the number is not greater than 10:

```
num = 5

if num > 10:
    print("Number is greater than 10")
else:
    print("Number is not greater than 10")
```

In this case:

- ***num > 10*** is the condition.
- Since num is 5 (which is not greater than 10), the condition ***num > 10*** evaluates to ***False***. Therefore, the code block under else **(print("Number is not greater than 10"))** is executed.

Key points to remember:

- Use if followed by a condition to check if it's true or false.
- Use else to specify a block of code to be executed if the condition is false.

The if-else statement provides a straightforward way to **create branching logic in your Python programs**, allowing different paths to be taken based on the evaluation of conditions.

5.3 Elif statement

In addition to if and else, Python also provides an **elif statement**, which stands for **"else if"**.

The elif statement **allows you to check multiple conditions sequentially**.

It is used when you have more than two possible outcomes to consider.

Here's how the elif statement works within the context of an if-elif-else structure:

```
if condition1:
 # Code to execute if condition1 is true
 statement1
 statement2
elif condition2:
 # Code to execute if condition2 is true
 statement3
 statement4
else:
 #Code to execute if all conditions are false
 statement5
 statement6
```

- **condition1, condition2, etc.:** These are expressions that evaluate to either *True or False*. The if statement checks condition1. **If condition1 is True**, the corresponding block of code is executed. **If condition1 is False**, the elif statement checks condition2, and so on.
- **else:** This code block is optional. It is **executed only if all preceding conditions** (condition1, condition2, etc.) **are False**.

For example, let's use an **if-elif-else statement** to determine the grade based on a student's score:

```
score = 85

if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
else:
    print("Grade F")
```

In this example:

- **score >= 90** is the first condition. If True, it prints "**Grade A**".
- **score >= 80** is the second condition. If True, it prints "**Grade B**".
- score >= 70 is the third condition. If True, it prints "**Grade C**".
- If none of the conditions (score >= 90, score >= 80, score >= 70) are True, the else block is executed, printing "**Grade F**".

Key point to remember:

- Use if to check the first condition, elif to check subsequent conditions, and else (if needed) to handle cases where all previous conditions are False.

The elif statement allows you to create complex decision-making processes in your Python programs, making it easy to handle multiple possible outcomes based on different conditions.

5.4 Nested if statement

A nested if statement is an ***if statement that is placed inside another if statement***. This allows you to test for multiple conditions, where each condition is checked only if the previous condition was true.

Here's how nested if statements work:

```
if condition1:
 # Code to execute if condition1 is true
 statement1
 statement2

 if condition2:
 # Code to execute if both condition1 and
condition2 are true
 statement3
 statement4
```

- **condition1:** This is the outer if statement condition. If condition1 is True, the code block under it (statement1, statement2, etc.) is executed.
- Inside the block of condition1, there is another if statement (if condition2). This is the nested if statement. It is only checked if condition1 is True. If condition2 is also True, the code block under if condition2 (statement3, statement4, etc.) is executed.

For example, let's use a nested if statement to check if a number is positive, and if so, whether it is even or odd:

```
num = 10

if num > 0:
 print("Number is positive")

 if num % 2 == 0:
 print("Number is even")
 else:
 print("Number is odd")
```

In this example:

- num > 0 is the outer condition. Since num is 10 (which is greater than 0), the outer if statement condition is True, so it prints "Number is positive".
- Inside the outer if block, there is a nested if statement (if num % 2 == 0). Since num is even (num % 2 == 0 is True), it prints "Number is even".

Key points to remember:

- Nested if statements allow you to check for multiple conditions in a hierarchical manner.
- Each if statement (including nested ones) requires proper indentation to denote its scope.
- Using nested if statements can help you handle more complex decision-making scenarios in your Python programs.

Nested if statements are useful when you need to perform different checks based on previous conditions being true, allowing for more specific and detailed logic in your code.

Chapter 6

LOOPS IN PYTHON

python™

6.1 While Loop

Python has two primary types of loop constructs: the **`while`** loop and the **`for`** loop.

A while loop in Python repeatedly executes a code block as long as a **given condition is true**. This allows you to perform repetitive tasks until a certain condition is met.

Here's how the while loop works:

```
while condition:
# Code to execute repeatedly as long as the
condition is true
 statement1
 statement2
 # ...
```

condition: This is an expression that evaluates to either True or False. The while loop will continue to execute the block of code as long as the condition remains True. Once the condition becomes False, the loop stops.

For example, let's use a while loop to print numbers **from 1 to 5**:

```
num = 1

while num <= 5:
    print(num)
    num += 1 # Increment num by 1
```

In this example:

- num <= 5 is the condition.
- Initially, num is 1. The loop checks if num <= 5 is True (which it is), so it prints the value of num and then increments num by 1 (num += 1).
- This process repeats, printing numbers 2, 3, 4, and 5. When num becomes 6, the condition num <= 5 becomes False, and the loop stops.

Key points to remember:

- Use a while loop when you need to repeat a block of code as long as a specific condition is true.
- Ensure the loop condition will eventually become False to avoid an infinite loop. This typically involves modifying a variable within the loop (like num += 1 in the example).

6.2 For loop

A for loop in Python is **used to iterate over a sequence** (such as a **list, tuple, string, or range**) and execute a code block for each item in the sequence. This allows you to perform repetitive tasks with ease.

Here's how the for loop works:

```
for item in sequence:
 # Code to execute for each item in the sequence
 statement1
 statement2
 # ...
```

- **item:** This is a variable that takes the value of each item in the sequence, one at a time.
- **sequence:** This is the sequence of items you want to iterate over. It can be a list, tuple, string, range, or any other iterable.

For example, let's use a for loop to print each item in a list:

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

- fruit is the variable that will take the value of each item in the fruits list, one at a time.
- The for loop iterates over the fruits list and prints each fruit: "apple", "banana", and "cherry".

Key points to remember:

- Use a for loop to iterate over a sequence of items.
- The for loop automatically takes each item from the sequence and assigns it to the loop variable (item in the first example, num in the second example).

The for loop is fundamental for performing repetitive tasks in Python, making it easy to work with each item in a sequence and execute code accordingly.

6.2.1 For loop using range() function

The range() function in Python generates a sequence of numbers, which can be used with a for loop to iterate over a specific range of values.

This is particularly useful when you want to repeat a block of code a certain number of times.

Here's how the for loop works with the range() function:

```
for variable in range(start, stop, step):
 # Code to execute for each number in the
range
 statement1
 statement2
 # ...
```

- **start:** The starting value of the sequence (inclusive). If omitted, it defaults to 0.
- **stop:** The ending value of the sequence (exclusive).
- **step:** The amount by which the sequence increments. If omitted, it defaults to 1.

For example, let's use a for loop with range() to print numbers from 0 to 4:

```
for num in range(5):
    print(num)
```

In this example:

- range(5) generates a sequence of numbers from 0 to 4.
- The for loop iterates over this sequence, and num takes each value (0, 1, 2, 3, 4) one by one, printing each number.

You can also specify a starting value and a step value. For example, let's print even numbers from 2 to 10:

```
for num in range(2, 11, 2):
 print(num)
```

In this example:

- range(2, 11, 2) generates a sequence of numbers starting from 2, up to (but not including) 11, incrementing by 2 each time.
- The for loop iterates over this sequence, and num takes each value (2, 4, 6, 8, 10) one by one, printing each number.

You can also use the range() function to count backward by specifying a negative step value. For example, let's print numbers from 5 to 1:

```
for num in range(5, 0, -1):
 print(num)
```

In this example:

- range(5, 0, -1) generates a sequence of numbers starting from 5, down to (but not including) 0, decrementing by 1 each time.
- The for loop iterates over this sequence, and num takes each value (5, 4, 3, 2, 1) one by one, printing each number.

Key points to remember:

- Use the range() function with a for loop to generate a sequence of numbers.
- The start parameter is optional and defaults to 0.
- The step parameter is optional and defaults to 1.

6.2.2 Nested for loop

A nested for loop is a for loop inside another for loop. This allows you to perform repeated iterations over multiple sequences or dimensions.

Nested loops are commonly used for working with multi-dimensional data structures like lists of lists (e.g., matrices).

Here's how nested for loops work:

```
for variable1 in sequence1:
 # Code to execute for each item in sequence1
 statement1
 statement2
 # ...

 for variable2 in sequence2:
 # Code to execute for each item in sequence2
 statement3
 statement4
 # ...
```

- variable1 iterates over sequence1.
- Inside the outer loop, variable2 iterates over sequence2 for each value of variable1.

For example, let's use a nested for loop to print each element in a 2-dimensional list (matrix):

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for item in row:
        print(item, end=' ')
    print()
```

In this example:

- The outer loop iterates over each row in the matrix.
- The inner loop iterates over each item in the current row.
- print(item, end=' ') prints each item in a row on the same line.
- print() prints a newline after each row to format the matrix correctly.

So, the output will be:

1 2 3
4 5 6
7 8 9

Here's another example, demonstrating a nested for loop to generate a multiplication table:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i * j}\t", end="")
    print()
```

In this example:

- The outer loop iterates i from 1 to 5.
- The inner loop iterates j from 1 to 5 for each value of i.
- print(f"{i * j}\t", end="") prints the product of i and j followed by a tab space on the same line.

The output will be:

1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25

Key points to remember:

- Nested for loops are used to iterate over multi-dimensional sequences.
- The outer loop runs first, and for each iteration of the outer loop, the inner loop runs completely.

Nested for loops are powerful for handling complex data structures and performing operations that require multiple levels of iteration.

Chapter 7

STRING OPERATIONS (ADVANCED)

python™

In this chapter, we will cover **advanced string operations**. We have already seen a basic understanding of strings, including their definition, modification, and common methods.

7.1 String Indexing and Slicing

String indexing allows you to access individual characters in a string using their positions (indices). In Python, string indices start from 0 for the first character and go up to ‘len(string) - 1' for the last character. Negative indices can also be used, starting from -1 for the last character and going backwards.

Here's how string indexing works:

```
string = "Hello, World!"
```

- string[0] gives the first character: 'H'
- string[7] gives the eighth character: 'W'
- string[-1] gives the last character: '!'
- string[-5] gives the fifth character from the end: 'o'

String Slicing

String slicing allows you to access a substring by specifying a range of indices. The syntax for slicing is **string[start:stop:step]**.

- **start:** The starting index of the slice (inclusive). If omitted, defaults to the beginning of the string.
- **stop:** The ending index of the slice (exclusive). If omitted, defaults to the end of the string.
- **step:** The step size (optional). If omitted, defaults to 1.

For Example:

```
string = "Hello, World!"
```

1. Slice from the beginning to a specific index:

string[:5] gives the first five characters: 'Hello'

2. Slice from a specific index to the end:

string[7:] gives the substring from the eighth character to the end: 'World!'

3. Slice between two specific indices:

string[7:12] gives the substring from the eighth to the twelfth character: 'World'

4. Slice with a specific step size:

string[::2] gives every second character: 'Hlo ol!'
string[1:10:2] gives every second character from the second to the tenth character: 'el,Wr'

Key Points to Remember:

- String indexing allows access to individual characters using positive and negative indices.
- String slicing allows extraction of substrings using the start:stop:step syntax.
- Omitting start, stop, or step uses default values (beginning of string, end of string, step size of 1, respectively).
- Proper understanding of indexing and slicing helps manipulate and access specific parts of strings efficiently.

String indexing and slicing are powerful tools in Python, making it easy to work with and manipulate strings in various ways.

7.2 String Formatting

String formatting allows you to create strings with dynamic content by inserting values into placeholders. Python offers several ways to format strings, including the % operator, the format() method, and f-strings (formatted string literals).

Overview of Different Methods

1. Percent % Operator:

- The **%** operator is the oldest method, similar to **C-style string formatting.**
- Example: **"Hello, %s!" % name**

2. format() Method:

- The **format()** method is more flexible and readable than the **%** operator.
- Example: **"Hello, {}!".format(name)**

3. F-strings (Formatted String Literals):

- F-strings, introduced in **Python 3.6,** are the most modern and convenient way to format strings.
- Example: **f"Hello, {name}!"**

Detailed Explanation of the format() Method

The **format()** method allows you to insert values into **placeholders {}** within a string. You can use positional or keyword arguments to specify the values to be inserted.

Basic Usage:

```
name = "Alice"
age = 30

# Positional arguments
formatted_string = "Hello, {}! You are {}
years old.".format(name, age)

print(formatted_string)
# Output: Hello, Alice! You are 30 years old.

# Keyword arguments
formatted_string = "Hello, {name}! You are
{age} years old.".format(name=name, age=age)

print(formatted_string)
# Output: Hello, Alice! You are 30 years old.
```

Indexing and Reordering

You can use indices to reorder the values:

```
formatted_string = "{1}, {0}".format("first",
"second")

print(formatted_string)
# Output: second, first
```

Formatting Types

You can also format numbers, dates, and other data types:

```
# Formatting numbers
number = 123.456

formatted_string = "Number:
{:.2f}".format(number) # Two decimal places
print(formatted_string)
# Output: Number: 123.46
```

Detailed Explanation of F-strings

F-strings (formatted string literals) provide a concise and readable way to format strings. They start with an f or F before the opening quote and use curly braces {} to embed expressions.

Basic Usage

```
name = "Alice"
age = 30

formatted_string = f"Hello, {name}! You are {age} years old."

print(formatted_string)
# Output: Hello, Alice! You are 30 years old.
```

Expressions Inside F-strings

```
formatted_string = f"Next year, you will be {age + 1} years old."

print(formatted_string)
# Output: Next year, you will be 31 years old.
```

Formatting Types

You can also format numbers and other data types within f-strings:

```
number = 123.456

formatted_string = f"Number: {number:.2f}"
# Two decimal places

print(formatted_string)
# Output: Number: 123.46
```

Examples and Use Cases

Example 1: Formatting a Greeting Message

1) Using **format() method:**

```
name = "Bob"
greeting = "Hello, {}! Welcome to the
club.".format(name)

print(greeting)
# Output: Hello, Bob! Welcome to the club.
```

2) Using **f-string:**

```
name = "Bob"
greeting = f"Hello, {name}! Welcome to the
club."

print(greeting)
# Output: Hello, Bob! Welcome to the club.
```

Example 2: Formatting a Table

1) Using **format() method:**

```
data = {"name": "Alice", "age": 30, "city":
"New York"}

formatted_string = "Name: {name}, Age: {age},
City: {city}".format(**data)

print(formatted_string)
# Output: Name: Alice, Age: 30, City: New York
```

2) Using **f-strings:**

```
name = "Alice"
age = 30
city = "New York"

formatted_string = f"Name: {name}, Age: {age},
City: {city}"
print(formatted_string)
# Output: Name: Alice, Age: 30, City: New York
```

Example 3: Number Formatting

1) Using **format() method:**

```
number = 1234.56789
formatted_string = "Number:
{:.2f}".format(number) # Two decimal places

print(formatted_string)
# Output: Number: 1234.57
```

2) Using **f-strings:**

```
number = 1234.56789
formatted_string = f"Number: {number:.2f}"
# Two decimal places

print(formatted_string)
# Output: Number: 1234.57
```

Key Points to Remember

- The % operator is an older method for string formatting.
- The format() method offers flexibility and readability.
- F-strings (introduced in Python 3.6) are the most modern and convenient way to format strings, allowing for embedding expressions directly within strings.
- Proper string formatting improves code readability and helps dynamically generate strings with variable content.

7.3 Common String Operations

Python provides a rich set of built-in string methods that allow you to perform various operations on strings. These methods can help you manipulate, search, and transform strings effectively.

Overview of Commonly Used String Operations

1. **Changing Case:**

- **upper():** Converts all characters in the string to uppercase.
- **lower():** Converts all characters in the string to lowercase.
- **capitalize():** Converts the first character to uppercase and the rest to lowercase.
- **title():** Converts the first character of each word to uppercase.

2. **Trimming Whitespace:**

- **strip():** Removes leading and trailing whitespace.
- **lstrip():** Removes leading whitespace.
- **rstrip():** Removes trailing whitespace.

3. Searching and Replacing:

- **find():** Returns the index of the first occurrence of a substring.
- **rfind():** Returns the index of the last occurrence of a substring.
- **replace():** Replaces all occurrences of a substring with another substring.

4. Splitting and Joining:

- **split():** Splits the string into a list of substrings based on a delimiter.
- **join():** Joins a list of strings into a single string with a specified delimiter.

5. Checking String Properties:

- **startswith():** Checks if the string starts with a specified substring.
- **endswith():** Checks if the string ends with a specified substring.
- **isalpha():** Checks if all characters in the string are alphabetic.
- **isdigit():** Checks if all characters in the string are digits.

Key Points to Remember

- String methods in Python provide a powerful way to manipulate and analyze strings.
- Use methods like upper(), lower(), strip(), find(), replace(), split(), and join() to perform common string operations.
- Methods like startswith(), endswith(), isalpha(), and isdigit() help check properties of strings.
- Refer to chapter **'Detailed Overview of Python Data Types'** for detailed explanations and additional string methods.

Chapter 8

FUNCTIONS IN PYTHON

python™

8.1 Functions in Python

Functions in Python are blocks of reusable code that perform a specific task. They help organize code into manageable sections, making it easier to read, debug, and maintain.

1. Purpose: Functions allow you to write code once and use it multiple times throughout your program. This avoids repetition and makes your code cleaner.

2. Structure: A function typically takes input, processes it, and returns an output. The input is called parameters or arguments, and the output is called the return value.

3. Benefits:

- **Modularity:** Functions break your program into smaller, manageable pieces.
- **Reusability:** You can use the same function in different parts of your program without rewriting the code.
- **Maintainability:** Changes in the function's logic need to be made only once, making the program easier to update and debug.

4. Examples of Built-in Functions:

- **print():** Outputs data to the console.
- **len():** Returns the length of a sequence like a string, list, or tuple.

5. User-defined Functions: While Python provides many built-in functions, you can also create your own to perform specific tasks unique to your program. This allows for greater flexibility and customization.

8.2 Creating functions, function calling

1. Creating Function

There are four parts involved in creating a function in Python. Let's look at each part clearly and easily.

- **Define the Function:** Use the def keyword followed by the function name and parentheses ().
- **Add Parameters (Optional):** Inside the parentheses, you can define the parameters the function will take.
- **Function Body:** Indent the lines of code that make up the function's body. This is where you put the code you want the function to execute.
- **Return Statement (Optional):** Use the return keyword to return a value from the function.

For Example:

```
def greet(name):
# Function definition with one parameter
    print(f"Hello, {name}!") # Function body
```

2. Calling a Function

To call a function, write its name followed by parentheses (). If the function has parameters, pass the arguments inside the parentheses.

For Example:

```
greet("Alice")
# Calling the greet function with "Alice" as the argument
```

8.3 Return statement

The return statement in Python is used in a function to send a value back to the place where the function was called.

Purpose of the return Statement

- **Send Back a Value:** It allows a function to produce a result that can be used elsewhere in the program.
- **Exit the Function:** When a return statement is executed, the function stops running and exits immediately.

How It Works

- **Syntax:** The return statement is followed by the value or expression you want to return.
- **Multiple Returns:** You can have multiple return statements in a function, but only one will be executed, depending on the function's logic.

Examples:

1. Returning a Single Value:

```
def add(a, b):
        return a + b # returns the sum of a & b
result = add(3, 4) # result will be 7
print(result) # This prints 7
```

2. Returning Multiple Values:

```
def get_name_and_age():
    name = "Alice"
    age = 30
    return name, age # This returns a tuple

name, age = get_name_and_age()
# name will be "Alice", age will be 30

print(name, age)
# This prints "Alice 30"
```

3. Conditional Return:

```
def check_even_or_odd(number):
    if number % 2 == 0:
        return "Even" # Returns "Even"
    else:
        return "Odd" # Returns "Odd"

result = check_even_or_odd(7)
# result will be "Odd"
print(result) # This prints "Odd"
```

8.4 Scope of variables

The scope of a variable in Python refers to the region of the code where the variable is accessible. Understanding scope helps you know where a variable can be used and where it cannot. There are two main types of scope: local and global.

Local Scope

- Variables defined inside a function are in the local scope.
- They can only be accessed within that function.
- They exist only while the function is executing. Once the function finishes, the local variables are destroyed.

For Example:

```
def my_function():
    x = 10 # x is a local variable
    print(x) # This will print 10

my_function()

print(x)
# This will raise an error because x is not
accessible outside the function
```

Global Scope

- Variables defined outside any function are in the global scope.
- They can be accessed anywhere in the code after they are defined.
- They exist for the lifetime of the program.

For Example:

```
y = 20 # y is a global variable

def another_function():
        print(y) # This will print 20 because y
is accessible inside the function

another_function()

print(y)
# This will print 20 because y is accessible
outside the function
```

Global Variables Inside Functions

- If you want to modify a global variable inside a function, you need to use the global keyword.

For Example:

```
z = 30 # z is a global variable

def modify_global_variable():
    global z
    z = 40 # modifies the global variable z

modify_global_variable()

print(z)
# This will print 40 because z was modified
inside the function
```

Local Variables with the Same Name as Global Variables

- If a local variable has the same name as a global variable, the local variable will shadow the global variable within the function.

For Example:

```
a = 50 # a is a global variable
def shadowing_example():
 a = 60 # a is a local variable that shadows
the global variable
 print(a)
# This will print 60, the local variable

shadowing_example()

print(a)
# This will print 50, the global variable
```

Nonlocal Scope

- Variables defined in the nearest enclosing scope that is not global.
- They can be accessed in nested functions using the nonlocal keyword.

For Example:

```
def outer_function():
    b = 70 # b is in the nonlocal scope
    def inner_function():
        nonlocal b
        b = 80 # This modifies the nonlocal
variable b
        print(b) # This will print 80
    inner_function()
    print(b) # This will print 80

outer_function()
```

8.5 Lambda Functions in Python

Lambda functions in Python are small, anonymous functions defined using the lambda keyword. They are also known as lambda expressions and are often used for short, simple operations where a full function definition is not necessary.

Syntax

The syntax for a lambda function is:

```
lambda arguments: expression
```

- **lambda:** The keyword used to define a lambda function.
- **arguments:** A comma-separated list of parameters (can be zero or more).
- **expression:** A single expression that is evaluated and returned.

Characteristics

- Lambda functions do not have a name.
- They are limited to a single expression, not a block of statements.

Examples

1. Basic Lambda Function:

```
add = lambda x, y: x + y
print(add(3, 5)) # This prints 8
```

2. Using Lambda with map:

```
numbers = [1, 2, 3, 4, 5]
squared = map(lambda x: x**2, numbers)
print(list(squared))
# This prints [1, 4, 9, 16, 25]
```

3. Using Lambda with filter:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = filter(lambda x: x % 2 == 0, numbers)
print(list(evens))
# This prints [2, 4, 6, 8, 10]
```

4. Using Lambda with sorted:

```
points = [(1, 2), (4, 1), (5, -1), (2, 3)]
sorted_points = sorted(points, key=lambda
point: point[1])
print(sorted_points)
# This prints [(5, -1), (4, 1), (1, 2), (2, 3)]
```

Use Cases

- When you need a small function for a short period and don't want to define a full function.
- Often used with functions like map(), filter(), and sorted(), which take other functions as arguments.

Limitations

- Lambda functions can only contain one expression, limiting their complexity.
- While they can make code shorter, overusing them can make code harder to read.

Comparison with Regular Functions

1. Lambda Function:

```
square = lambda x: x**2
print(square(4)) # Prints 16
```

2. Regular Function:

```
def square(x):
    return x**2
print(square(4)) # Prints 16
```

8.6 Arrays in Python

An array is a special variable that can hold more than one value at a time. If you have a list of items (a list of bike names, for example), storing the bikes in a single variable could look like this:

```
bike1 = "Harley"
bike2 = "Ducati"
bike3 = "Yamaha"
```

Instead, you can use an array to store all the bike names in one variable.

Accessing Elements of an Array

To access the elements of an array, you can use their index. Array indices start from 0.

Example: Get the value of the first array item

```
bikes = ["Harley", "Ducati", "Yamaha"]
x = bikes[0] # x will be "Harley"
```

Modifying Elements of an Array

You can change the value of an array element by accessing it via its index.

```
bikes[0] = "Kawasaki"
# Now the first item is "Kawasaki"
```

Length of an Array

Use the len() function to return the length of an array.

```
length = len(bikes) # length will be 3
```

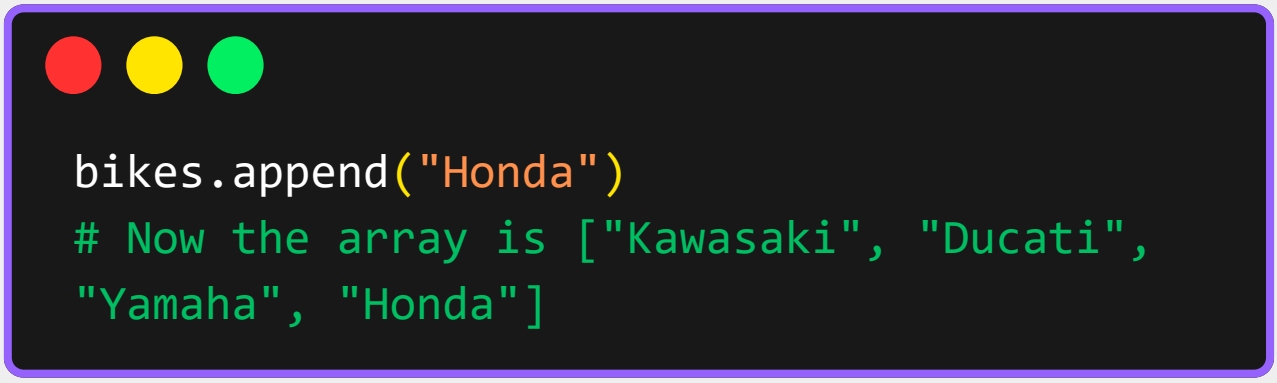
Looping Through Array Elements

Use a for loop to iterate through all the elements of an array.

```
for bike in bikes:
    print(bike)
```

Adding Array Elements

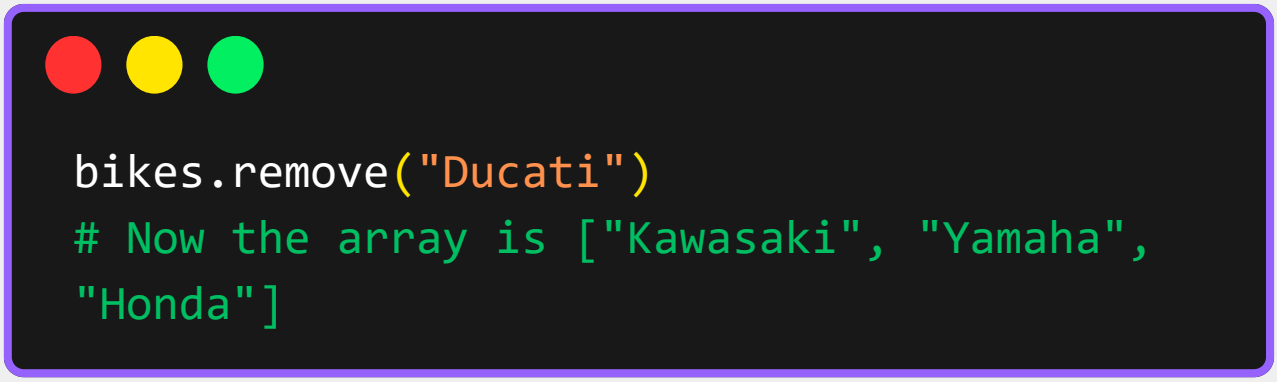
Use the append() method to add an element to an array.



```
bikes.append("Honda")
# Now the array is ["Kawasaki", "Ducati",
"Yamaha", "Honda"]
```

Removing Array Elements

Use the remove() method to delete an element from the array.



```
bikes.remove("Ducati")
# Now the array is ["Kawasaki", "Yamaha",
"Honda"]
```

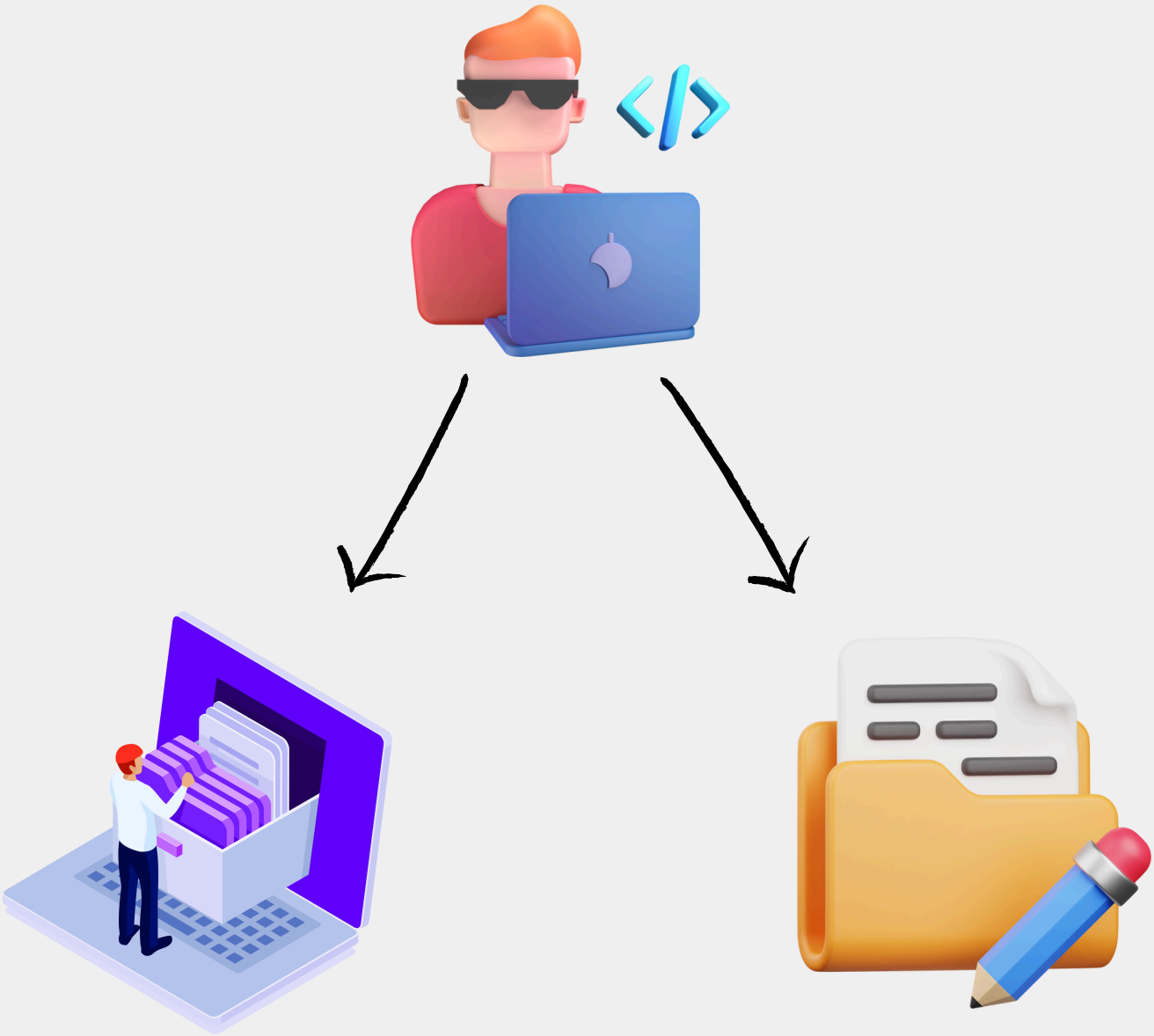
Chapter 9

FILE HANDLING IN PYTHON

python™

9.1 Introduction to file input/output

In Python, file input/output (I/O) allows you to read data from files and write data to files. This is useful for storing information, loading data, and sharing results. You can work with text files and binary files depending on the data you need to handle.



9.2 Opening and Closing Files

To work with files in Python, you first need to open them. The open() function is used to open a file. After you're done, you should close the file to free up resources.

1. Opening a file:

```
file = open('example.txt', 'r')
# 'r' means read mode
```

2. Closing a file:

```
file.close()
```

Using the **with** statement ensures that files are properly closed after their suite finishes, even if an exception is raised.

```
with open('example.txt', 'r') as file:
 content = file.read()
# File is automatically closed here
```

9.3 Reading and Writing Text Files

Text files contain readable characters. You can read from and write to these files using various methods.

1. Reading

- **read():** Reads the entire file.
- **readline():** Reads one line at a time.
- **readlines():** Reads all lines into a list.

```
with open('example.txt', 'r') as file:
    content = file.read()
```

2. Writing

- **write():** Writes a string to the file.
- **writelines():** Writes a list of strings to the file.

```
with open('example.txt', 'w') as file:
    file.write('Hello, World!')
```

9.4 Working with Binary Files

Binary files contain data in binary form (0s and 1s). You use binary mode to read and write such files.

- **Opening in binary mode:**

```
with open('example.bin', 'rb') as file:
# 'rb' means read binary
 content = file.read()
```

- **Writing in binary mode:**

```
with open('example.bin', 'wb') as file:
# 'wb' means write binary
    file.write(b'\x00\x01\x02\x03')
```

9.5 Exception Handling in File Operations

When working with files, errors can occur (e.g., file not found, permission denied). Use try-except blocks to handle these exceptions gracefully.

```
try:
    with open('example.txt', 'r') as file:
        content = file.read()
except FileNotFoundError:
    print("The file was not found.")
except IOError:
    print("An error occurred while accessing the file.")
```

This structure ensures your program can handle errors and continue running smoothly.

Chapter 10

OBJECT-ORIENTED PROGRAMMING (OOP)

python™

10.1 Introduction to OOP in Python

Object-Oriented Programming (OOP) is a way of organizing and designing your code using objects, which can represent real-world things or concepts. OOP helps make your code more modular, reusable, and easier to understand.

10.2 Classes and Objects

1. Classes:

A class is a blueprint for creating objects. It defines a set of attributes and methods that the objects created from the class will have.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print("Woof!")
```

2. Objects:

An object is an instance of a class. It represents a specific entity.

```
my_dog = Dog("Buddy", 3)
print(my_dog.name) # Output: Buddy
my_dog.bark()   # Output: Woof!
```

10.3 Constructors and Destructors

1. Constructors:

The __init__ method is called a constructor. It initializes the object's attributes when the object is created.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

2. Destructors:

The __del__ method is called a destructor. It is called when an object is about to be destroyed. It's not commonly used, but it can be helpful for cleanup actions.

```
class Dog:
    def __del__(self):
        print(f"{self.name} is being
destroyed.")
```

10.4 Inheritance and Polymorphism

1. Inheritance:

Inheritance allows a class (child class) to inherit attributes and methods from another class (parent class). This helps to reuse and extend existing code.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

my_dog = Dog("Buddy")
print(my_dog.speak()) # Output: Woof!
```

2. Polymorphism:

Polymorphism allows methods to do different things based on the object it is acting upon, even if they share the same name.

```
class Cat(Animal):
    def speak(self):
        return "Meow!"

animals = [Dog("Buddy"), Cat("Whiskers")]

for animal in animals:
    print(animal.speak())
# Output: Woof! Meow!
```

10.5 Encapsulation and Data Hiding

1. Encapsulation:

Encapsulation means bundling the data (attributes) and methods (functions) that operate on the data into a single unit (class). It helps to keep the data safe from outside interference and misuse.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age  # Private attribute

    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age > 0:
            self.__age = age
```

2. Data Hiding:

Data hiding restricts direct access to some of an object's attributes. This is done by prefixing the attribute name with an underscore (_).

```
person = Person("Alice", 30)

print(person.get_age()) # Output: 30

person.set_age(35)

print(person.get_age()) # Output: 35
```

10.6 Method Overriding and Overloading

1. Method Overriding:

Method overriding allows a child class to provide a specific implementation of a method that is already defined in its parent class.

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"

my_dog = Dog()
print(my_dog.speak()) # Output: Woof!
```

2. Method Overloading:

Python does not support traditional method overloading like some other languages (where the same method name can have different arguments). Instead, you can use default arguments or variable-length arguments.

```python
class Math:
    def add(self, a, b, c=0):
        return a + b + c

math = Math()
print(math.add(1, 2))  # Output: 3
print(math.add(1, 2, 3)) # Output: 6
```

Chapter 11

EXCEPTION HANDLING IN PYTHON

python™

11.1 Introduction to Exception Handling

Exception handling in Python is a way to deal with errors that occur during the execution of a program. Instead of the program crashing, you can handle these errors gracefully, providing alternative actions or informative messages to the user.

11.2 Exception Handling Mechanism

In Python, you use the try, except, else, and finally blocks to handle exceptions.

- **try:** Write the code that might cause an exception inside the try block.
- **except:** Write the code to handle the exception inside the except block.
- **else:** This block runs if no exceptions occur in the try block.
- **finally:** This block runs no matter what, even if an exception occurs or not.

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ZeroDivisionError:
    print("You cannot divide by zero!")
except ValueError:
    print("Invalid input! Please enter a
number.")
else:
    print("The result is:", result)
finally:
    print("This will always be executed.")
```

11.3 Handling Multiple Exceptions

You can handle multiple exceptions by specifying multiple except blocks. This allows you to respond differently to various types of errors.

Example:

```
try:
 number = int(input("Enter a number: "))
 result = 10 / number
except(ZeroDivisionError, ValueError) as e:
 print(f"An error occurred: {e}")
```

You can also handle multiple exceptions in a single except block:

```
try:
 number = int(input("Enter a number: "))
 result = 10 / number
except(ZeroDivisionError, ValueError):
 print("Either a ZeroDivisionError or
ValueError occurred.")
```

11.4 Custom Exceptions

Sometimes, you might need to create your own exceptions to handle specific situations in your program. You can do this by defining a new class that inherits from the built-in Exception class.

Example:

```
class CustomError(Exception):
    def __init__(self, message):
        self.message = message

def check_value(value):
    if value < 0:
        raise CustomError("Negative values are
not allowed!")

try:
    check_value(-5)
except CustomError as e:
    print(f"A custom error occurred:
{e.message}")
```

11.5 Error Handling Strategies

When handling errors, it's important to follow good strategies to make your code robust and maintainable.

1. Be specific with exceptions: Catch specific exceptions rather than a generic Exception. This makes your code clearer and easier to debug.

```
try:
    # some code
except ZeroDivisionError:
    # handle division by zero
except ValueError:
    # handle value error
```

2. Provide informative error messages: Give clear and helpful error messages to the user so they understand what went wrong and how to fix it.

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid input! Please enter a valid number.")
```

3. Clean up resources: Use the finally block to release resources, like closing files or network connections, even if an error occurs.

```
try:
    file = open('example.txt', 'r')
    # work with file
except IOError:
    print("An error occurred while accessing
the file.")
finally:
    file.close()
```

4. Log errors: Logging errors can help you keep track of issues in your application for future debugging.

```
import logging

logging.basicConfig(filename='app.log',
level=logging.ERROR)

try:
    # some code
except Exception as e:
    logging.error(f"An error occurred: {e}")
```